

# 1.ÜNİTE - YAPAY ZEKA, MAKİNE ÖĞRENMESİ VE İLİŞKİLİ TEMEL KAVRAMLAR

Yapay zekanın tarihsel gelişimini, temel türlerini ve modern teknolojiadaki yerini ele almaktadır. Yapay zekanın bir alt dalı olan makine öğrenmesinin çalışma prensipleri, geleneksel programlamadan farkları ve temel öğrenme stratejileri (danışmanlı, danışmansız, pekiştirmeli) detaylandırılmaktadır. Ayrıca veri madenciliği süreci ve veri analitiği türleri arasındaki farklar açıklanmaktadır.

## Yapay Zeka (AI) ve Tarihsel Gelişimi

Yapay zeka, makinelerin insan zekasına özgü akıl yürütme, öğrenme ve problem çözme gibi yetenekleri sergilemesini hedefleyen bir alandır.

- **Tanım:** İnsan zekası gerektiren görevleri (görsel algı, dil çevirisi, karar verme vb.) yerine getirebilen bilgisayar sistemlerinin teorisidir.
- **Tarihsel Dönemeçler:** 1950'lerde Alan Turing'in "Makineler düşünebilir mi?" sorusu ve Turing Testi ile başlayan süreç, 1956 Dartmouth Konferansı ile bilim dalı kimliği kazanmıştır.
- **Yapay Zeka Kışları:** Beklentilerin karşılanamadığı ve desteklerin kesildiği 1974-1980 ve 1987-1994 dönemleri "yapay zeka kışı" olarak adlandırılır.
- **Modern Dönem:** 1990 sonrasında veri madenciliği ve derin öğrenmenin yükselişiyle karmaşık sorunlar çözülmeye başlanmış; günümüzde ise metin, resim ve müzik üretebilen "Üretken Yapay Zeka" (Generative AI) aşamasına gelinmiştir.

## Yapay Zeka Türleri

Yapay zeka sistemleri, yeteneklerine ve kapsamlarına göre üç ana kategoriye ayrılır:

- **Dar (Zayıf) Yapay Zeka:** Sadece belirli bir görevi (örneğin sadece satranç oynamak veya sadece yüz tanımak) yerine getirebilir. Genelleme yeteneği yoktur; bir alandaki bilgisini başka alana aktaramaz.
- **Güçlü (Genel) Yapay Zeka:** Bir insanın yapabileceği her türlü entelektüel görevi yerine getirebilen, öz farkındalığa sahip sistemleri ifade eder.

- **Süper Yapay Zeka:** İnsan zekasını her alanda (problem çözme, yaratıcılık, bilgelik) büyük ölçüde aşan kuramsal bir zeka türüdür.

## Makine Öğrenmesi (ML) ve Temel Bileşenleri

Makine öğrenmesi, açıkça programlanmak yerine verilerden ve deneyimlerden öğrenerek performansını artıran algoritmalar bütünüdür.

- **Dört Temel Unsur:**

1. **Görev (Task):** Sistemin gerçekleştirmesi istenen spesifik iş (örneğin teşhis veya tahmin).
2. **Deneyim (Experience):** Sistemin öğrenmesi için kullanılan "veri".
3. **Algoritma:** Veriyi işleyerek öğrenmeyi sağlayan matematiksel yol.
4. **Performans:** Modelin ne kadar iyi öğrendiğini ölçen kriterler (doğruluk, kesinlik vb.).

- **Geleneksel Programlama Farkı:** Klasik programlamada kurallar insan tarafından yazılır ve veri girilerek cevap alınır. Makine öğrenmesinde ise veri ve cevaplar sisteme verilir; sistem bu ikisi arasındaki "kuralları" kendi keşfeder.

## Makine Öğrenmesi Stratejileri

Makinelerin öğrenme biçimleri, verinin niteliğine ve hedeflenen çıktıya göre üçe ayrılır:

- **Danışmanlı (Supervised) Öğrenme:** Etiketlenmiş veriler (girdi-çıkı çiftleri) kullanılır. Hedef değişken kategorikse **Sınıflandırma**, nümerikse **Regresyon** olarak adlandırılır.
- **Danışmansız (Unsupervised) Öğrenme:** Etiketlenmiş veri yoktur. Veri içindeki gizli desenleri ve grupları bulmayı hedefler. En temel örneği **Kümeleme** analizidir.
- **Pekiştirmeli (Reinforcement) Öğrenme:** Bir ajanın çevreyle etkileşime girerek "ödül-ceza" mekanizmasıyla en iyi stratejiyi (politika) öğrenmesidir. Oyunlar ve robotik alanında yaygındır.

## Veri Madenciliği ve Veriden Bilgi Keşfi (KDD)

Büyük veri yığınları içerisinde değerli ve anlamlı bilgilerin çıkarılması sürecidir.

- **Süreç Adımları:** Veri temizleme, bütünleştirme, seçim, dönüştürme, veri madenciliği, örüntü değerlendirme ve bilgi gösterimi aşamalarından oluşur.
- **ML ile İlişkisi:** Veri madenciliği gizli kalıpları keşfetmeye odaklanırken, makine öğrenmesi bu kalıpları kullanarak geleceğe yönelik tahmin modelleri oluşturmaya odaklanır.

## Veri Analitiği Yaklaşımları

Analitik çalışmalar, cevap aranan soruya göre üç grupta incelenir:

- **Açıklayıcı Analitik:** "Ne oldu?" sorusuna yanıt arar. Geçmiş verileri özetler ve görselleştirir.
- **Tahminsel Analitik:** "Ne olacak?" sorusuna odaklanır. Geçmiş verilerden yola çıkarak gelecek öngörülerini oluşturur.
- **Kuralcı Analitik:** "Nasıl olmalı?" sorusuna yanıt verir. Belirli bir hedefe ulaşmak için eylem planları ve öneriler sunar.

# 2.ÜNİTE - MAKİNE ÖĞRENMESİ SÜRECİ

Bir makine öğrenmesi projesinin başlangıcından uygulama aşamasına kadar geçen yapılandırılmış süreci ele almaktadır. Süreç, endüstri standardı olan **CRISP-DM** (Veri Madenciliği için Çapraz Endüstri Standardı Süreç Modeli) adımları üzerinden açıklanmaktadır. Temel odak noktası; problemin doğru tanımlanması, verinin kalitesinin artırılması (ön işleme) ve model performansının bilimsel yöntemlerle ölçülmesidir.

## CRISP-DM ve Problemin Tanımlanması

Makine öğrenmesi süreci, rastgele işlemler yerine belirli bir metodoloji çerçevesinde yürütülmelidir.

- **CRISP-DM Adımları:** Süreç; problemin tanımlanması, veriyi anlama, veri hazırlama, modelleme, değerlendirme ve uygulama olmak üzere altı temel aşamadan oluşur.

- **Problem Çerçeveleme:** Projenin başında hedeflerin ve başarı kriterlerinin net bir şekilde ortaya konması, sonraki tüm teknik kararları (veri seçimi, algoritma türü vb.) doğrudan etkiler.
- **Uygulama Örnekleri:** Tıbbi görüntülerden hastalık teşhisi, trafik yoğunluğu tahmini, kaza riski öngörüsü ve kişiselleştirilmiş ürün önerileri bu sürecin çıktıları olan tipik problemlerdir.

## Veri Elde Etme ve Veri Türleri

Modelin "deneyimi" olan verinin kalitesi, sonuçların başarısını doğrudan belirleyen "çöp-girdi, çöp-çıkıtı" prensibine dayanır.

- **Veri Kaynakları:** Analizlerde UCI, Kaggle ve Google Dataset Search gibi kamuya açık platformlardan veya yapılandırılmış (tablo) ve yapılandırılmamış (ses, görüntü) kurumsal kaynaklardan veri toplanabilir.
- **Sayısal Veri:** Kendi içinde tam sayılarla ifade edilen **ayrık** (çocuk sayısı gibi) ve her değeri alabilen **sürekli** (sıcaklık, boy gibi) olarak ikiye ayrılır.
- **Kategorik Veri:** Sadece iki seçeneği olan **ikili** (evet/hayır), belirli bir hiyerarşisi olan **sıralı** (eğitim durumu) ve hiyerarşisi olmayan **nominal** (renkler) türlerini kapsar.

## Veri Ön İşleme ve Hazırlama

Ham veri genellikle hatalı veya eksik olduğundan, modelleme öncesinde temizlenmeli ve analizlere uygun formata getirilmelidir.

- **Eksik Değer Yönetimi:** Eksik veriler; sayısal niteliklerde ortalama/medyan ile, kategorik niteliklerde ise en sık tekrar eden değer (mod) ile tamamlanabilir.
- **Normalizasyon:** Farklı ölçeklerdeki sayısal verilerin (örneğin yaş ve gelir) modele eşit katkıda bulunması için [0, 1] gibi standart bir aralığa çekilmesidir.
- **Dummy (Yapay) Kodlama:** Kategorik verilerin bilgisayarın anlayabileceği sayısal (0 ve 1) değerlere dönüştürülmesidir.
- **Boyut Azaltma:** Analiz karmaşıklığını ve "çok boyutluluğun lanetini" engellemek için gereksiz niteliklerin veri setinden çıkarılmasıdır.

## Modelleme ve Performans Değerlendirme

Eğitilen modelin gerçek dünya verileri üzerinde ne kadar başarılı olacağı farklı yöntemlerle test edilir.

- **Veri Ayırma Yöntemleri:**

- **Hold-out (İkili Ayırma):** Verinin bir kısmının (%70-80) eğitim, kalanının test için ayrılmasıdır.
- **Çapraz Geçerleme (Cross-Validation):** Verinin farklı katlara ayrılıp her katın sırayla test edilmesiyle daha güvenilir performans sonuçları elde edilmesidir.

- **Sınıflandırma Ölçütleri:** Karmaşıklık matrisi kullanılarak hesaplanan **Doğruluk**, **Duyarlılık** (pozitifleri bulma başarısı), **Kesinlik** ve **F1-Skoru** gibi metriklerle ölçülür.

- **Regresyon Ölçütleri:** Hata payını ölçen **MAE** (Ortalama Mutlak Hata), **MSE** (Ortalama Karesel Hata) ve **RMSE** gibi değerler kullanılır.

- **Kümeleme Değerlendirmesi:** En uygun küme sayısını belirlemek için **Dirsek Yöntemi** (SSE bükülme noktası) veya kümelerin ayrışma kalitesini gösteren **Silhouette Katsayısı** (-1 ile 1 arası) kullanılır.

## 3.ÜNİTE - PYTHON İLE VERİ GÖRSELLEŞTİRME

Veri görselleştirme, ham verinin grafiksel bir temsile dönüştürülerek insan algısı için daha anlaşılır kılınması sürecidir. Makine öğrenmesinde bu süreç, veriyi modellemeye hazırlarken (Keşifsel Veri Analizi - EDA) hataları ayıklamak ve değişkenler arasındaki gizli bağları bulmak için kullanılır.

### 1. Görselleştirmenin Makine Öğrenmesindeki Rolü

Görselleştirme sadece bir sunum aracı değil, aynı zamanda bir analiz yöntemidir:

- **Karmaşıklığı Azaltma:** Binlerce satırlık veri tablolarını tek bir grafikte özetleyerek genel eğilimi görmemizi sağlar.
- **Anomali Tespiti:** Veri setindeki hatalı girişleri veya uç (aykırı) değerleri görsel olarak hızla saptamaya yardımcı olur.

- **Değişken İlişkileri:** Hangi özelliklerin (features) hedef değişkeni daha fazla etkilediğini anlamamıza yarar.

## 2. Python Görselleştirme Kütüphaneleri

Python, bu alanda üç temel kütüphane üzerinden ilerler:

- **Matplotlib:** En alt seviye ve en esnek kütüphanedir; her türlü grafik detayına müdahale imkanı verir.
- **Seaborn:** Matplotlib tabanlıdır ancak daha modern, estetik ve istatistiksel odaklı grafikler (ısı haritaları, keman grafikleri vb.) sunar.
- **Pandas Visualization:** Pandas kütüphanesi içindeki `plot()` fonksiyonu ile hızlı ve pratik görseller oluşturulmasını sağlar.

---

## 3. Temel Grafik Türleri ve Kullanım Amaçları

### A. Dağılım Analizi: Histogram ve Kutu Grafiği

Verinin tek bir nitelik bazında nasıl yayıldığını anlamak için kullanılır:

- **Histogram:** Belirli aralıklardaki veri frekansını sütunlarla gösterir. Verinin normal dağılıma (çan eğrisi) yakın olup olmadığını anlamak için kritiktir.
- **Kutu Grafiği (Box Plot):** Verinin beş sayı özetini (en küçük, Q1, medyan, Q3, en büyük) sunar. Kutunun dışındaki noktalar doğrudan "aykırı değer" olarak yorumlanır.

a box plot showing median, quartiles, and outliers, yapay zekayla üretilmiş

### B. İlişki Analizi: Serpilme Diyagramı (Scatter Plot)

İki sayısal değişken arasındaki korelasyonu inceler:

- **Pozitif/Negatif İlişki:** Eğer noktalar yukarı doğru bir hat izliyorsa pozitif, aşağı doğruysa negatif bir ilişki vardır.
- **Kümelenme:** Veri noktalarının belirli bölgelerde yoğunlaşması, veri içindeki alt gruplar hakkında ipucu verir.

### C. Zaman Serisi: Çizgi Grafiği (Line Plot)

Sürekli verilerin (genellikle zaman bazlı) değişimini izlemek içindir:

- X ekseninde zaman, Y ekseninde ise gözlemlenen değer yer alır.

- Trendleri (artış/azalış) ve mevsimsel döngüleri belirlemek için en uygun grafik türüdür.

#### D. Karşılaştırma: Sütun ve Pasta Grafiği

- **Sütun Grafiği (Bar Plot):** Kategorik değişkenlerin (örneğin şehirler veya eğitim seviyeleri) miktarlarını karşılaştırmak için kullanılır.
- **Pasta Grafiği (Pie Chart):** Bir niteliğin sınıflarının bütün içindeki payını (yüzdelik olarak) gösterir. Kategori sayısı çok fazlaysa okunabilirliği azaldığı için dikkatli kullanılmalıdır.

---

## 4. Görselleştirme Süreci (İş Akışı)

Python ile bir görsel oluştururken genellikle şu adımlar izlenir:

1. **Veri Hazırlama:** Eksik verilerin temizlenmesi ve görselleştirilecek sütunların seçilmesi.
2. **Figür Oluşturma:** Grafiğin çizileceği tualin (figure) ve eksenlerin tanımlanması.
3. **Çizim:** İlgili kütüphane fonksiyonu (örn: `plt.scatter()`) ile verinin grafiğe aktarılması.
4. **Özelleştirme:** Başlık, eksen isimleri, lejant (açıklama) ve renklerin eklenmesi.
5. **Kaydetme/Gösterme:** Çıktının ekrana yansıtılması veya dosya olarak kaydedilmesi.

## 4.ÜNİTE - PYTHON İLE VERİ ÖN-İŞLEME

---

Ham verilerin makine öğrenmesi algoritmaları tarafından kullanılmadan önce geçmesi gereken kritik hazırlık aşamalarını kapsamaktadır. Ham veriler genellikle eksik, gürültülü veya tutarsızdır; veri ön-işleme bu sorunları gidererek modelin doğruluğunu ve güvenilirliğini artırır. Süreç temel olarak veri temizleme, veri dönüştürme ve veriyi bölme adımlarından oluşur.

## Veri Keşfi ve İlk Bakış

Analize başlamadan önce verinin yapısını anlamak için Python'un **Pandas** kütüphanesi kullanılır.

- **Veri Yükleme:** `read_csv()` veya `read_excel()` fonksiyonları ile farklı formatlardaki veriler Python ortamına aktarılır.
- **Genel İnceleme:** `head()` fonksiyonu verinin ilk satırlarını, `info()` fonksiyonu veri tiplerini ve bellek kullanımını, `describe()` ise temel istatistiksel özetleri (ortalama, standart sapma vb.) sunar.
- **Boyut Kontrolü:** `shape` özelliği ile veri setindeki toplam satır ve sütun sayısı belirlenir.

## Eksik Veri Yönetimi

Gerçek dünya veri setlerinde sıklıkla karşılaşılan boş (NaN/null) değerlerin yönetilmesi sürecidir.

- **Tespit:** `isnull().sum()` komutu ile her bir sütunda kaç adet eksik değer olduğu raporlanır.
- **Silme:** Eksik verinin çok az olduğu durumlarda `dropna()` ile ilgili satır veya sütunlar veri setinden çıkarılabilir.
- **Doldurma (Imputation):** Eksik değerler; sayısal sütunlarda **ortalama (mean)** veya **medyan**, kategorik sütunlarda ise **en sık tekrar eden (mod)** değer ile doldurulur.

## Veri Dönüştürme Teknikleri

Algoritmaların veriyi doğru yorumlayabilmesi için yapılan matematiksel düzenlemelerdir.

- **Kategorik Verilerin Dönüştürülmesi:**
  - **Label Encoding:** Kategorik değerlere 0'dan başlayarak sayısal etiketler atar (örneğin: Düşük=0, Orta=1, Yüksek=2).
  - **One-Hot Encoding:** Her kategori için yeni bir sütun oluşturur ve ilgili kategori için 1, diğerleri için 0 değeri atar.
- **Özellik Ölçeklendirme (Feature Scaling):** Farklı aralıklardaki sayısal değerlerin birbiriyle uyumlu hale getirilmesidir.
  - **Normalizasyon (Min-Max):** Değerleri [0, 1] aralığına sıkıştırır.

- **Standartlaştırma (Z-Score):** Veriyi ortalaması 0, standart sapması 1 olacak şekilde dönüştürür.

## Veri Setinin Bölünmesi

Modelin başarısını tarafsız bir şekilde ölçmek için veri seti ikiye (veya üçe) ayrılır.

- **Eğitim Seti (Train Set):** Algoritmanın verideki desenleri öğrenmek için kullandığı ana bölümdür.
- **Test Seti (Test Set):** Modelin daha önce hiç görmediği veriler üzerindeki performansını ölçmek için kullanılan bölümdür.
- **Uygulama:** Python'da Scikit-learn kütüphanesi içindeki `train_test_split` fonksiyonu ile genellikle %70-80 eğitim, %20-30 test olacak şekilde rastgele bölünme yapılır.

# 5.ÜNİTE - K-ORTALAMALAR (K-MEANS) ALGORİTMASI

---

Veri setindeki gizli yapıları ve benzerlikleri keşfetmek için kullanılan K-ortalamlar algoritmasını ele almaktadır. K-ortalamlar, etiketlenmemiş verileri benzer özelliklerine göre belirli sayıda ( $k$ ) kümeye ayıran, bölme tabanlı bir kümeleme algoritmasıdır.

## 1. Algoritmanın Temel Mantığı ve İşleyişi

K-ortalamlar, verileri birbirine benzeyen gruplara ayırırken "küme içi varyansı en aza indirme" prensibiyle çalışır. Algoritmanın işleyiş adımları şu şekildedir:

- **Küme Sayısının Belirlenmesi:** Başlangıçta verinin kaç gruba ayrılacağı ( $k$ ) kullanıcı tarafından seçilir.
- **Merkez Atama:** Rastgele  $k$  adet nokta "küme merkezi" (centroid) olarak seçilir.
- **Atama Adımı:** Her bir veri noktası, kendisine en yakın olan küme merkezine atanır; bu mesafe genellikle Öklid uzaklığı ile hesaplanır.
- **Güncelleme Adımı:** Atama işlemi bittikten sonra, her kümenin yeni merkezi, o kümeye dahil olan noktaların ortalaması alınarak yeniden hesaplanır.

- **Döngü:** Küme merkezleri değişmeyene veya belirlenen bir yineleme sayısına ulaşılan kadar atama ve güncelleme işlemleri devam eder.

## 2. Kritik Başarı Faktörleri

Algoritmanın performansı ve sonuçların doğruluğu iki temel etkene bağlıdır:

- **Başlangıç Noktaları:** Rastgele seçilen ilk merkezler, algoritmanın nihai sonucunu etkileyebilir. Farklı başlangıç noktalarıyla algoritmayı tekrar çalıştırmak yaygın bir yaklaşımdır.
- **Uzaklık Ölçümü:** Veri noktalarının benzerliğini ölçmek için en sık **Öklid uzaklığı** kullanılır; ancak verinin yapısına göre Manhattan veya Minkowski gibi farklı metrikler de tercih edilebilir.

## 3. En Uygun Küme Sayısının (\$k\$) Seçimi

Veri setinin kaç kümeye ayrılacağı her zaman önceden bilinmeyebilir. Bu durumda şu tekniklerden faydalanılır:

- **Dirsek (Elbow) Yöntemi:** Farklı \$k\$ değerleri için küme içi hata kareler toplamı (SSE) hesaplanır. SSE değerindeki düşüşün yavaşladığı "dirsek" noktası en uygun \$k\$ değeri olarak seçilir.
- **Silhouette Analizi:** Küme içi benzerlik ile kümeler arası uzaklığı kıyaslayan bir katsayıdır. 1'e yakın değerler kümelemenin başarılı olduğunu gösterir.

## 4. Algoritmanın Avantajları ve Sınırlılıkları

- **Avantajlar:** Büyük veri setlerinde hızlı çalışır, uygulanması ve anlaşılması kolaydır.
- **Sınırlılıklar:** \$k\$ değerinin kullanıcı tarafından girilmesi gerekir, aykırı değerlere (outliers) karşı hassastır ve dairesel olmayan (farklı geometrik şekilli) kümeleri ayırmakta zorlanabilir.

## 5. Uygulama Alanları

K-ortalamlar algoritması gerçek dünyada birçok farklı amaçla kullanılır:

- **Müşteri Segmentasyonu:** Müşterileri alışveriş alışkanlıklarına göre gruplandırma.
- **Görüntü Sıkıştırma:** Benzer renk piksellerini gruplayarak görüntü boyutunu küçültme.
- **Anomali Tespiti:** Genel kümelenmenin dışında kalan verileri saptama.

# 6.ÜNİTE - BASİT VE ÇOKLU DOĞRUSAL REGRESYON ANALİZİ

Bağımlı bir değişken ile bir veya daha fazla bağımsız değişken arasındaki matematiksel ilişkiyi modelleyen doğrusal regresyon tekniklerini ele almaktadır. Regresyon analizi, verideki eğilimleri anlamak ve gelecekteki değerleri öngörmek (tahmin etmek) amacıyla kullanılır.

## 1. Regresyon Analizinin Temel Kavramları

Regresyon analizi, değişkenler arasındaki ilişkinin yönünü ve şiddetini belirlemeyi hedefler.

- Bağımlı Değişken ( $y$ ):** Tahmin edilmek istenen, sonuç niteliğindeki nümerik değişkendir (örneğin: ev fiyatı).
- Bağımsız Değişken ( $x$ ):** Bağımlı değişkeni etkilediği varsayılan tahmin edici değişkenlerdir (örneğin: evin metrekaresi).
- Doğrusallık:** Değişkenler arasındaki ilişkinin bir doğru boyunca ilerlediği varsayılır.

## 2. Basit Doğrusal Regresyon

Yalnızca bir bağımsız değişkenin bağımlı değişken üzerindeki etkisinin incelendiği modeldir.

- Model Formülü:**  $y = \beta_0 + \beta_1 x + \epsilon$  şeklinde ifade edilir.
  - $\beta_0$ : Sabit terim (doğrunun y eksenini kestiği nokta).
  - $\beta_1$ : Eğim katsayısı (bağımsız değişkendeki 1 birimlik artışın bağımlı değişkene etkisi).
  - $\epsilon$ : Hata terimi.
- En Küçük Kareler Yöntemi (OLS):** Gerçek değerler ile tahmin edilen değerler arasındaki farkların (artıkların) kareler toplamını en aza indirerek en iyi uyum doğrusunu bulmayı sağlar.

### 3. Çoklu Doğrusal Regresyon

Bağımlı değişkenin iki veya daha fazla bağımsız değişken tarafından açıklandığı modellerdir.

- **Model Formülü:**  $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_nx_n + \epsilon$  şeklindedir.
- **Kullanım Alanı:** Gerçek dünya problemlerinde sonuçlar genellikle birden fazla faktöre bağlıdır (örneğin: evin fiyatı metrekare, oda sayısı ve konuma bağlıdır).

### 4. Regresyon Modelinin Başarı Ölçütleri

Kurulan modelin veriyi ne kadar iyi açıkladığını belirlemek için şu ölçütler kullanılır:

- **Belirlilik Katsayısı ( $R^2$ ):** Bağımsız değişkenlerin, bağımlı değişkendeki değişimin yüzde kaçını açıkladığını gösterir. 1'e yakın olması modelin başarısının yüksek olduğunu ifade eder.
- **Düzeltilmiş  $R^2$ :** Modele gereksiz değişken eklendiğinde yanıltıcı artışları engelleyerek daha sağlıklı bir başarı ölçüsü sunar.
- **Hata Metrikleri:** Tahminlerin gerçek değerlerden sapmasını ölçen **MSE** (Ortalama Karesel Hata) ve **RMSE** (Kök Ortalama Karesel Hata) değerlerinin düşük olması beklenir.

### 5. Temel Varsayımlar

Sağlıklı bir regresyon analizi için verinin şu şartları sağlaması gerekir:

- **Doğrusallık:** Değişkenler arasında doğrusal bir ilişki olmalıdır.
- **Normal Dağılım:** Hata terimleri normal dağılım göstermelidir.
- **Çoklu Doğrusal Bağlantı Problemi (Multicollinearity):** Bağımsız değişkenler arasında kendi aralarında çok güçlü bir ilişki olmamalıdır; aksi halde model katsayıları istikrarsızlaşır.

## 7.ÜNİTE - K-EN YAKIN KOMŞU (k-NN) ALGORİTMASI

---

"Bana arkadaşını söyle, sana kim olduğun söyleyeyim" prensibiyle çalışan, örnek tabanlı bir öğrenme algoritması olan k-NN'i ele almaktadır. Algoritma, yeni bir veri noktasını tahmin ederken mevcut veri setindeki en yakın benzerlerine bakar.

## 1. Algoritmanın Temel Çalışma Prensibi

k-NN, verilerin özellik uzayındaki konumlarını temel alan non-parametrik bir algoritmadır.

- **Örnek Tabanlı Öğrenme:** Algoritma, eğitim aşamasında bir model kurmaz; bunun yerine tüm eğitim verilerini hafızada tutar. Bu nedenle "tembel öğrenci" (lazy learner) olarak da adlandırılır.
- **Sınıflandırma Mantığı:** Yeni bir örnek geldiğinde, ona en yakın  $k$  adet komşu bulunur. Bu komşular arasında en çok görülen sınıf, yeni örneğin sınıfı olarak atanır (çoğunluk oylaması).
- **Regresyon Mantığı:** Tahmin edilecek değer, en yakın  $k$  komşunun hedef değerlerinin ortalaması alınarak bulunur.

## 2. Kritik Parametreler ve Seçimler

Algoritmanın başarısı iki temel seçime doğrudan bağlıdır:

- **k Değerinin Seçimi:** \* Çok küçük  $k$  değerleri (örneğin  $k=1$ ), modelin gürültüye karşı hassas olmasına ve aşırı öğrenmeye (overfitting) neden olabilir.
  - Çok büyük  $k$  değerleri, sınıflar arasındaki sınırların belirsizleşmesine yol açabilir.
  - Genellikle tek sayılar seçilerek oylamada eşitlik durumu engellenir.
- **Uzaklık Ölçümleri:** Veri noktaları arasındaki benzerliği hesaplamak için farklı metrikler kullanılır:
  - **Öklid Uzaklığı:** En yaygın kullanılan kuş uçuşu mesafe ölçüsüdür.
  - **Manhattan Uzaklığı:** Eksenler boyunca gidilen yolların toplamıdır.
  - **Minkowski Uzaklığı:** Öklid ve Manhattan'ın genel bir formudur.

### 3. Veri Ön-İşlemenin Önemi

k-NN algoritması, uzaklık temelli çalıştığı için verinin ölçeğine karşı son derece hassastır.

- **Normalizasyon Gerekliliği:** Eğer bir nitelik 0-1 aralığında, diğeri ise 0-10.000 aralığındaysa, büyük değerlere sahip nitelik uzaklık hesabını domine eder. Bu yüzden verilerin mutlaka ölçeklendirilmesi (Min-Max veya Standartlaştırma) gerekir.
- **Boyut Azaltma:** Nitelik sayısı arttıkça "çok boyutluluğun laneti" nedeniyle hesaplama maliyeti artar ve benzerlik ölçümü zorlaşır.

### 4. Algoritmanın Avantajları ve Dezavantajları

- **Avantajları:** Anlaşılması ve uygulanması kolaydır, karmaşık matematiksel varsayımlar gerektirmez, çok sınıflı problemlerde başarılıdır.
- **Dezavantajları:** Büyük veri setlerinde tahmin aşaması yavaştır (çünkü her seferinde tüm verilerle uzaklık hesaplar), bellek kullanımı yüksektir, gürültülü verilere ve eksik değerlere karşı duyarlıdır.

## 8.ÜNİTE - NAIVE BAYES ALGORİTMASI

---

Temelini Thomas Bayes'in olasılık kuramından alan Naive Bayes algoritmasını ele almaktadır. Algoritma, bir örneğin belirli bir sınıfa ait olma olasılığını hesaplayarak sınıflandırma yapar.

### 1. Bayes Teoremi ve "Naive" Yaklaşımı

Algoritmanın çalışma prensibi koşullu olasılık hesaplamalarına dayanır.

- **Bayes Teoremi:** Bir olayın gerçekleşme olasılığını, o olayla ilgili önceden bilinen koşullara dayanarak hesaplar.
- **"Naive" (Saf/Safdil) Varsayımı:** Algoritma, sınıftaki her bir özelliğin (niteliğin) birbirinden tamamen bağımsız olduğunu varsayar. Gerçek hayatta özellikler arasında genellikle bir ilişki olsa da, bu basitleştirme algoritmanın çok hızlı ve etkili çalışmasını sağlar.

## 2. Algoritmanın Türleri

Veri setindeki niteliklerin dağılımına göre farklı Naive Bayes türleri kullanılır:

- **Gaussian Naive Bayes:** Sürekli sayısal veriler için kullanılır ve verilerin normal (Gauss) dağılımına sahip olduğu varsayılır.
- **Multinomial Naive Bayes:** Genellikle metin sınıflandırmada (belge türü belirleme gibi) tercih edilir; kelime sayımları gibi ayrık verilerle çalışır.
- **Bernoulli Naive Bayes:** Özelliklerin sadece "var" veya "yok" (1 veya 0) gibi ikili (binary) değerler aldığı durumlarda kullanılır.

## 3. Uygulama Alanları

Naive Bayes, özellikle yüksek boyutlu verilerde ve metin madenciliğinde popülerdir:

- **Spam Filtreleme:** Bir e-postanın içindeki kelimelerin olasılıklarına bakarak "spam" veya "güvenli" olarak sınıflandırılması.
- **Duygu Analizi:** Müşteri yorumlarının pozitif veya negatif olarak kategorize edilmesi.
- **Haber Sınıflandırma:** Haber metinlerinin konularına (spor, siyaset, teknoloji) göre ayrıştırılması.

## 4. Avantajlar ve Sınırlılıklar

- **Avantajları:** Eğitilmesi ve tahmin yapması oldukça hızlıdır. Az miktarda eğitim verisiyle bile iyi sonuçlar verebilir. Karmaşık modellerin aksine hesaplama maliyeti düşüktür.
- **Sınırlılıkları:** Özelliklerin bağımsız olduğu varsayımı gerçek hayatta her zaman tutmaz, bu durum modelin doğruluğunu etkileyebilir. Eğer eğitim setinde olmayan bir kategori test setinde karşımıza çıkarsa "Sıfır Olasılık" (Zero Frequency) sorunu oluşur (bu durum Laplace düzeltmesi ile aşılar).

# 9.ÜNİTE - KARAR AĞAÇLARI (DECISION TREES)

---

Karar ağaçları, verileri belirli özelliklere göre dallara ayırarak hiyerarşik bir yapı oluşturan danışmanlı bir öğrenme algoritmasıdır. Görselleştirilmesi ve

yorumlanması son derece kolay olduğu için iş dünyasında ve tıbbi teşhislerde sıklıkla tercih edilir.

## 1. Temel Yapı ve Kavramlar

Bir karar ağacı akış şemasına benzer bir yapı sergiler:

- **Kök Düğüm (Root Node):** Ağacın en tepesindeki düğümdür ve veriyi ilk bölen en önemli özelliği temsil eder.
- **İç Düğümler (Decision Nodes):** Bir özellik üzerindeki test sonucuna göre dallanmanın devam ettiği noktalardır.
- **Yaprak Düğümler (Leaf Nodes):** Dallanmanın bittiği ve nihai kararın (sınıf veya değer) verildiği uç noktalardır.
- **Dallar (Branches):** Bir düğümden çıkan ve testin sonucuna göre gidilecek yolu gösteren bağlantılardır.

## 2. Bölünme Kriterleri (Bilgi Kazancı)

Algoritma, veriyi en "saf" (homojen) gruplara ayırmak için matematiksel kriterler kullanır:

- **Entropi:** Verideki belirsizliğin veya düzensizliğin ölçüsüdür.
- **Gini Safsızlığı (Gini Index):** Bir düğümdaki verilerin rastgele seçildiğinde yanlış etiketlenme olasılığını ölçer.
- **Bilgi Kazancı (Information Gain):** Bir özelliği kullanarak veriyi böldüğümüzde entropide ne kadar azalma olduğunu gösterir.

## 3. Karar Ağacı Uygulama Örneği

Notlarınızdaki mantığa dayanarak, bir bankanın kredi onay sürecini yöneten basit bir karar ağacı örneği şu şekildedir:

**Soru: Müşteriye Kredi Verilmeli mi?**

1. **Kök Düğüm:** Kredi Puanı > 700 mü?

◦ **EVET:** Gelir > 20.000 TL mi?

■ **EVET: KREDİ ONAYLANDI** (Yaprak)

■ **HAYIR: KREDİ REDDEDİLDİ** (Yaprak)

◦ **HAYIR:** Mevcut Borç < 5.000 TL mi?

■ **EVET: KREDİ ONAYLANDI** (Yaprak)

■ **HAYIR: KREDİ REDDEDİLDİ** (Yaprak)

#### 4. Avantajlar ve Dezavantajlar

- **Avantajları:** Veri hazırlama süreci (normalizasyon vb.) gerektirmez, hem sayısal hem kategorik verilerle çalışabilir ve sonuçlar tamamen şeffaftır.
- **Dezavantajları:** Verideki küçük değişikliklere karşı çok hassastır ve ağaç çok derinleşirse "aşırı öğrenme" (overfitting) sorunu yaşanabilir. Bu sorunu aşmak için "Budama" (Pruning) tekniği kullanılır.

## 10.ÜNİTE - YAPAY SİNİR AĞLARI (YSA)

Yapay Sinir Ağları, biyolojik sinir sisteminin bilgi işleme yöntemini taklit eden, birbirine bağlı işlem birimlerinden (yapay nöronlar) oluşan bir hesaplama modelidir. Doğrusal olmayan karmaşık ilişkileri öğrenebilme yeteneği sayesinde ses tanıma, görüntü işleme ve doğal dil işleme gibi alanlarda devrim yaratmıştır.

### 1. Yapay Nöronun Yapısı

Biyolojik bir nöronun temel işlevleri, yapay bir hücrede matematiksel olarak modellenir:

- **Girdiler (\$x\$):** Dış dünyadan veya önceki katmandan gelen verilerdir.
- **Ağırlıklar (\$w\$):** Her girdinin çıktı üzerindeki etkisini belirleyen katsayılardır; öğrenme süreci aslında bu ağırlıkların güncellenmesidir.
- **Toplama Fonksiyonu:** Girdilerin ağırlıklarıyla çarpılıp toplandığı (ve varsa bir sapma/bias değerinin eklendiği) bölümdür.
- **Aktivasyon Fonksiyonu:** Toplam değer belirlenir bir eşiği geçip geçmediğini kontrol eder ve çıktıyı üretir (Sigmoid, ReLU, Tanh gibi).

### 2. Ağ Mimarisi (Katmanlar)

YSA, nöronların belirli bir düzende dizildiği üç ana katmandan oluşur:

1. **Giriş Katmanı:** Verinin ağa dahil olduğu, herhangi bir işlemin yapılmadığı ilk katmandır.

2. **Gizli Katman(lar):** Giriş ve çıkış katmanları arasında yer alan, asıl öğrenmenin ve karmaşık hesaplamaların gerçekleştiği bölümdür. Derin öğrenme, çok sayıda gizli katmana sahip ağları ifade eder.
3. **Çıkış Katmanı:** Ağın ürettiği nihai tahminin (sınıf veya değer) sunulduğu katmandır.

### 3. Öğrenme Süreci

YSA'nın eğitimi iki temel aşamada gerçekleşir:

- **İleri Yayılım:** Veri girişten girer, ağırlıklarla işlenir ve bir tahmin üretilir.
- **Geri Yayılım (Backpropagation):** Üretilen tahmin ile gerçek değer arasındaki "hata" hesaplanır. Bu hata, ağ boyunca geriye doğru yayılır ve ağırlıklar hatayı azaltacak şekilde (optimizasyon algoritmalarıyla) güncellenir.

### 4. Karar Ağacı ile Karşılaştırmalı Örnek

Yapay Sinir Ağları, Karar Ağaçları gibi net "Evet/Hayır" kurallarıyla çalışmaz; bunun yerine verideki olasılıksal desenleri öğrenir.

#### Örnek: Rakam Tanıma

- **Karar Ağacı Yaklaşımı:** "Üstte bir yuvarlak var mı? Ortada bir çizgi var mı?" gibi katı kurallar dener.
- **YSA Yaklaşımı:** Görüntüdeki her bir pikseli girdi olarak alır. Gizli katmanlarda kenarları, köşeleri ve kavisleri "ağırlıklar" üzerinden öğrenir. Sonuçta, belirli bir piksel diziliminin %98 olasılıkla "8" rakamı olduğuna karar verir.

### 5. Avantajlar ve Sınırlılıklar

- **Avantajları:** Doğrusal olmayan ilişkileri çözebilir, gürültülü verilere karşı dayanıklıdır ve kendi kendine öğrenebilir.
- **Sınırlılıklar:** "Kara kutu" (black-box) niteliğindedir; bir karara tam olarak nasıl ulaştığını anlamak zordur. Çok fazla işlem gücü ve veri gerektirir.